CommitmentOS demo video script (rev 5)

Demo video — final consolidated script and production plan (rev 5)

Target runtime **3:55**. RULES VERIFIED (Devpost page, 2026-08-20): the requirement is "**~ 4-min Demo video**" (approximate — no hard cutoff stated) containing the problem overview, the value proposition, the app in action, and — mandatory — a demonstration that "the backend is running on Google Cloud (ie: Google Cloud Console, Cloud Run dashboard, Vertex AI logs, URL of .run, etc)". This script satisfies that three ways: the `.run.app` address bar throughout the take, the Cloud Logging cutaway, and the Cloud Run console shot. No hosting/visibility rule is stated for the demo video itself (the public-not-unlisted rule applies to the BONUS content piece); public upload stays the safe default. The judging rubric asks verbatim for "a live, unedited demo … and visible proof it runs on Google Cloud" (Demo & Production Readiness, 30%). Deadline: Aug 31, 2026 @ 5:00pm PDT.

Audience: judges with zero prior context. The one thing this video must do that nothing else can: **prove the real Gmail → Google Cloud → Calendar path executes**, in a continuous unedited span, with Google Cloud visible.

Editorial rule for every beat: **show the engineering; narrate the consequence.** The four memories a judge must leave with: it reads real commitments out of email · it refuses to guess (requests ≠ commitments, elapsed ≠ done) · it puts real work time on a real calendar · when the calendar changes, it fixes the plan itself in seconds. Everything else is a credibility enhancer, not the story.

Timeline

Time	Beat	Surface
0:00–0:22	Hook + what it is (≤58 words, ~160 wpm)	You / title card
0:22–2:47	**THE CONTINUOUS TAKE** (order below; final-take budget ≤2:25)	Gmail, dashboard, Calendar
2:47–2:54	Cloud Logging cutaway (three highlighted rows)	Cloud Logging
2:54–3:19	Sandbox: live Gemini on a novel message (precision claim as caption, not speech)	/sandbox (staged minutes before)
3:19–3:45	Five-node architecture + on-screen stat card	Diagram + Cloud Run console
3:45–3:55	Closing line + links	Title card

Inside the continuous take, in this order (the order is load-bearing):

1. Send the request email (counterparty account, clean compose popup)
2. Candidate appears on dashboard with its evidence quote
3. Accept by reply → same commitment updates, no duplicate

4. Confirm effort (type minutes) → planner runs → approve first plan
5. Google Calendar: the real blocks
6. **Completion beat on the pre-staged commitment** — introduced with the word "Separately"; give it a visually distinct title (e.g. new commitment "Q5 budget summary" vs staged "Finalize onboarding brief")
7. Quiet window (~20s): the audit-trust beat — NO calendar writes here; this protects the next step from push throttling
8. **Drop a meeting onto a reserved block → autonomous repair** (~9s)
9. Expand the plan_repaired audit entry: planner run, moved/preserved blocks, stable event ids, and the system's own measured `repair_latency_ms`

Script

Format (rev 5): **DO** lines are your hands — every tab switch, click, and keystroke, named at the exact moment it happens. **SAY** lines are your voice, worded for speaking rather than reading. NOTE lines are rehearsal and production reminders. The DO lines add no runtime — they name what you were already doing. Claims, guardrails, and timing budgets are unchanged from rev 4.

Account roles (the address mapping stays out of this public doc on purpose): **counterparty** = the separate account that sends the request email; **controlled** = the account CommitmentOS manages — its Gmail, Calendar, and dashboard are what's on camera.

0:00–0:22 — Hook + what it is

DO: You on camera, or voice-over your inbox; intro slides 1–3 carry the visuals (slide 2 doubles as the title card). ≤58 words at a calm ~160 wpm — do not inflate; the demo carries the rest.

SAY:

> "I make promises over email all the time — 'sure, I'll get you the deck by Friday' — and then it just sits in my inbox. My calendar has no idea the work exists.
>
> CommitmentOS finds those promises, books the work time on my Calendar, and repairs the plan itself when things change. Here it is deployed, running continuously against my real Gmail and Calendar."

NOTE: This wording survives the split-take fallback — the "no cuts" claim lives only in the caption over the actual continuous frames.

0:22–2:47 — THE CONTINUOUS TAKE

DO: Start ONE screen recording here and do not stop it until beat 9 is done. Address bar visible the whole time, browser at 110–125% zoom. The caption overlay (added in the edit) spans exactly these frames: "one continuous take — real accounts, deployed service". Final-take budget ≤2:25.

Your tab route through the take, in order: counterparty Gmail → dashboard → candidate detail → controlled Gmail → detail page → Calendar → dashboard (staged commitment) → Calendar → dashboard (audit) → Calendar → dashboard Activity.

1. Send the request

DO: Start in the **counterparty's Gmail**, compose popup already open — never show the inbox list.

SAY: "A colleague's asking me for something."

DO: Type and send: "*Hi — could you put together the Q5 budget summary? We need it by [day].*" — pick the day per the date-math note.

SAY: "And look at the deadline — just 'by [day]'. Plain human language."

2. Candidate appears

DO: Switch to the **dashboard tab** (Today view). Expect 10–30 s: push + interpretation + the 8 s poll. Never go silent over a static screen.

SAY: "Right now, Gmail is pushing that message to the agent…"

NOTE — filler if the wait runs long: "…Gmail sends a change signal to the service on Cloud Run, and the agent fetches just that one thread and reads it…"

SAY (the moment the candidate appears): "…there. And notice — it's not my commitment yet. It's a request, held as a candidate."

DO: Click **into the candidate** — its detail page shows the Source evidence card.

SAY: "…with the exact sentence it's based on. It won't invent obligations for me."

3. Accept by reply

DO: Switch to the **controlled account's Gmail** and reply on the thread: "*Yes, I'll take that on — you'll have it by [day] end of day.*" Send it.

DO: Switch back to the **commitment detail page** — the evidence stays visible, and the approval forms live here too.

SAY: "My reply updates the *same* commitment — it doesn't create a second one. One promise, one record… and now it's mine."

4. Effort + plan approval

DO: Stay on the detail page. The effort field is **blank** — that's the point. Click it and type **180**.

SAY: "It won't guess how long my work takes — I tell it. Three hours."

DO: Wait through the brief planner run until the plan card appears.

SAY: "…It's computing a plan against my real calendar right now… and here's its last *setup* question: may I write to your calendar? After this, in-policy repairs happen on their own — and anything outside policy still comes back to me."

DO: Click **Approve**.

NOTE: VERIFY IN REHEARSAL that both approvals surface on the detail page; fall back to Today's approval cards if they don't.

5. Real Calendar

DO: Switch to the **Google Calendar tab**.

SAY: "And there they are — real events, before the deadline, fitted around everything already on my calendar."

6. Completion beat

DO: Switch back to the **dashboard** and open the **pre-staged commitment** — its distinct title makes the switch visually unmistakable. Overlay caption: "Separate commitment — completion integrity".

SAY: "*Separately* — here's an older commitment, showing another control. One of its blocks elapsed this morning, and the agent did *not* assume the work happened. It's asking me."

DO: Type **60** into the check-in form and submit it.

SAY: "I log the sixty minutes I actually did… and the remaining work drops by exactly sixty."

DO: Click **Mark complete**, then its confirmation.

SAY: "And when I mark it done — it keeps my real number, and cancels the reserved time I no longer need."

DO: Flip to the **Calendar tab** — the future block is gone. Speak the next line over that view.

SAY: "Time passing never counts as progress. Only I do."

7. Quiet window — the trust beat

DO: Go back to the **dashboard** and expand the completion audit entry. Hold here ~20 s with NO calendar writes — this quiet window protects the next beat from push throttling. Point the cursor along the chain as you speak.

SAY: "Everything it does lands on an audit timeline. These entries connect the input it observed… the decision it made… the calendar action it wrote… and the verified result — all under one correlation id. You never have to take the agent's word for what happened."

8. THE CLIMAX — conflict → repair

DO: Switch to the **Calendar tab**.

SAY: "Now — the part I built this for. I'll drop a meeting right on top of the time it reserved. The kind of thing a coworker does to your week."

DO: Drag a new meeting **directly onto a reserved block**. Then take your hands off, visibly, and count through the wait (~9 s warmed).

SAY: "Hands off now… …there. It noticed through the calendar's own change feed, moved exactly that one block, and left the others untouched. I never clicked replan — the calendar change itself triggered the repair."

9. The receipt

DO: Switch to the **dashboard** → **Activity** tab and expand the `plan_repaired` entry. Read the moved/preserved counts and the latency FROM THE SCREEN, never from memory.

SAY: "And it can prove it: the planner run, the one block that moved, the ones it preserved, the stable event identity — and the repair latency it measured on itself: [the real number on screen] seconds."

NOTE — if the pause switch comes up at all, it is narration ONLY, never toggle it on camera: "And if I ever want it to stop — one switch pauses automatic action; anything in flight is held, not lost, and revalidated before it resumes."

2:47–2:54 — Cloud Logging cutaway

DO: Separate ~7 s clip — a visual receipt, not an explanation. Show the **Cloud Logging tab** (filter query already typed) with three rows highlighted, timestamps aligned: webhook in → reconciliation → calendar write. PRE-SCRUB the visible log lines for email addresses, ids, tokens, or query parameters before recording.

SAY: "Same story from Google Cloud's side — the webhook, the reconciliation, the calendar write. Seconds apart."

2:54–3:19 — Sandbox: prove the AI is live

DO: Separate take. Stage the session MINUTES before recording — sandbox worlds idle-expire and reads don't refresh the clock; a session staged in the morning will be dead by afternoon. Free-play lane chosen and subject entered BEFORE you roll; on camera you type only the message. The precision claim rides as an on-screen caption, not speech: "Same model · same prompt contract · isolated test data · no login".

SAY: "To show the interpretation happening live, I'll give the public sandbox a message it has never seen…"

DO: Type as the counterparty: "*Could you send me the revised onboarding guide by next Wednesday at 3pm?*" Send it, then expand the interpretation evidence.

SAY: "…and there it is, interpreted live by Gemini: the model, the wall-clock latency, and the exact words it cited."

NOTE: if the label shows `recorded-fallback`, that's a transient provider hiccup — wait ten minutes or reset, then re-record this segment.

3:19–3:45 — Architecture + the stat card

DO: Cut to the five-node diagram (`architecture_five_node.png`). The implementation nouns — idempotent tasks · transactional outbox · etag preconditions — appear as a caption strip, not speech. Then ~2 s of the **Cloud Run console** showing the service + revision. STAT CARD on screen, scoped exactly:

> **On-screen card:** 10 live end-to-end runs + 10 hardened seeded runs · 1,110 acceptance checks · 0 duplicate commitments or events · 73/73 security probes · ~$0.0008 per message

SAY: "It's one loop: Observe, Interpret, Plan, Policy, Execute — and every result gets observed again, so the loop closes. The rule that matters: Gemini only ever interprets language. Everything that touches the calendar is deterministic code, and etag preconditions reject stale writes — an older plan can't overwrite a newer calendar change. The receipts are on screen, and every number is in the repo, indexed to the exact deployment that produced it."

3:45–3:55 — Close

DO: Closing title card: tagline + the sandbox link, demo link, and repo link (canonical host).

SAY: "CommitmentOS doesn't plan once. It keeps the promise feasible as reality changes."

Pre-production (day before)

1. **Clean up, THEN stage** — run the audited controlled-account cleanup first; then create the pre-staged commitment (~120 min effort: one block that elapses the morning of recording + one later-in-week block that will visibly vanish on completion). Give it a title visually distinct from the new commitment's. Staging after cleanup, never before.
2. **Warm the instance**: `--min-instances 1` (verify actual current state; revert after recording).
3. **Verify**: monitoring ACTIVE and actions enabled (a paused monitor once silently killed a whole campaign); Gmail/Calendar watches current (renewal job is daily); zero stale pending approvals on Today; Calendar UI in Pacific.
4. **Date math**: choose the deadline expression for your recording day so it lands days out ("by Friday" from a Monday–Wednesday; otherwise "by next [day]"). More days = more visible placement spread.
5. **Accounts**: sign into `/app` fresh (12h session TTL) as the CONTROLLED account — the dashboard, the Gmail being read, and the Calendar on camera are all the controlled account's; its Gmail is also where you send the beat-3 acceptance reply from. The counterparty (your separate everyday address) sends the beat-1 request from its own browser profile — do NOT touch its OAuth (a wrong-account reconnect once poisoned a full night of runs). Set friendly display names on both accounts; the real controlled address WILL be on camera — deliberate choice, not an accident.
6. **Tabs prepped**: counterparty compose popup · dashboard · Calendar · Cloud Logging with the filter query already typed · Cloud Run console. Clean profile, notifications off, canonical host in the address bar (`commitmentos-2hscowvydq-uw.a.run.app`).
7. **Rehearse the full take twice, recorded at full quality** — a rehearsal where everything lands IS a usable final take. Keep every take.

Rehearsal pass criteria (two tiers)

- **Engineering pass** (the system works): push-to-candidate < 30 s; conflict-to-repair < 15 s; take completes < 2:45.
- **Final-take pass** (the footage fits the edit): all of the above AND the continuous take ≤ **2:25** (2:20 preferred, for edit tolerance). A take that passes engineering but runs long is a rehearsal, not a keeper.
- **Repair slower than ~15 s = failed FINAL take.** During rehearsal, keep rolling anyway — the once-a-minute safety reconciliation catches a suppressed push within ~60–75 s, and watching it happen is how you diagnose the delay. Footage of a 60 s repair is the emergency last resort only if no take ever passes.
- Two engineering failures → the **split fallback**: two continuous takes with the seam after beat 5 (Calendar view), each captioned honestly ("continuous — no cuts") — never cut inside beats 8–9. The intro makes no one-take claim at all, so the split needs no intro re-record.
- Verify during rehearsal: completing the staged commitment does NOT shift the new commitment's blocks (preserve-then-allocate should hold them).

Overlay & asset inventory (produce BEFORE recording day)

0. **PRODUCED** — intro slides 1–3 (`intro_slide_{1,2,3}.png`, 2x/1080p), timed to the intro narration: split-screen problem ("The promise exists. The time doesn't.") → name + Finds/Books/Repairs → "What you are about to see" transition. Slide 2 doubles as the opening title card.
1. Opening title card — covered by intro slide 2
2. Continuous-take caption: "one continuous take — real accounts, deployed service"
3. Completion overlay: "Separate commitment — completion integrity"
4. Sandbox caption: "Same model · same prompt contract · isolated test data · no login"
5. Implementation-nouns strip (architecture beat): "Idempotent tasks · Transactional outbox · Etag preconditions"
6. Stat card: "10 live end-to-end runs + 10 hardened seeded runs · 1,110 acceptance checks · 0 duplicate commitments or events · 73/73 security probes · ~$0.0008 per message"
7. Closing card: tagline + sandbox link, demo link, repo link (canonical host)

Recording notes

- 1080p minimum, browser at 110–125% zoom (embedded players compress), cursor visible.
- Live mic during the continuous take (authenticity through the waits); clean VO for intro/architecture/close is fine.
- No copyrighted music.
- Captions/subtitles recommended.

Post-recording

- Confirm monitoring active + actions enabled (mandatory if anything was toggled).
- Revert `--min-instances` if desired.
- Upload **public** on YouTube (confirm the rules, but public is the safe default for judging — an unlisted link risks a rule technicality), test playback on a cold browser, then put the link in Devpost and the LinkedIn post.

Truthfulness guardrails (non-negotiable)

- The "no cuts" caption spans exactly the frames that are continuous; the spoken intro makes no one-take claim, so it stays true under the fallback.
- Narrate what the viewer sees: YOU drop the conflicting meeting (don't say "someone books a meeting" while visibly dragging it yourself). A counterparty calendar invite is an UNTESTED conflict path — do not attempt it for the video.
- The repair-trigger line is "I never clicked replan — the calendar change itself triggered the repair" (states the mechanism); never "nobody opened an app" or "nobody opened CommitmentOS" — the interface is visibly open.
- Mechanisms, not absolutes: "etag preconditions reject stale writes," not "can never overwrite."
- The stat card scopes the campaigns explicitly: 10 live end-to-end + 10 hardened seeded. Scoped evidence reads stronger, not weaker.

- Never call `/demo` "live" — it is seeded; the sandbox and the continuous take are the live proof.
- Read the repair latency number that actually appears on screen.
- "Same model, same prompt contract" is the exact sandbox claim (production interpreter class and prompt on a sandbox-only key) — not "same system."
- **One description per surface, used consistently everywhere:**

Surface	Correct description
Real-account demo	Production service connected to real Gmail and Calendar
`/sandbox`	Public interactive environment — same model and prompt contract, isolated test data
`/demo`	Seeded, read-only product walkthrough

- The stat card carries the numbers; don't re-speak them — spoken narration stays at the two-ideas-plus-pointer level.